

UNIVERSITY OF TRIESTE
Department of Mathematics, Informatics and Geosciences

*Master's Degree in
Data Science & Artificial Intelligence*

Spatio-Temporal Logic Guided Optimization for Collective Adaptive Systems

Candidate
Annalisa Paladino

Supervisor
Prof. Francesca Cairoli

A.Y. 2024/2025

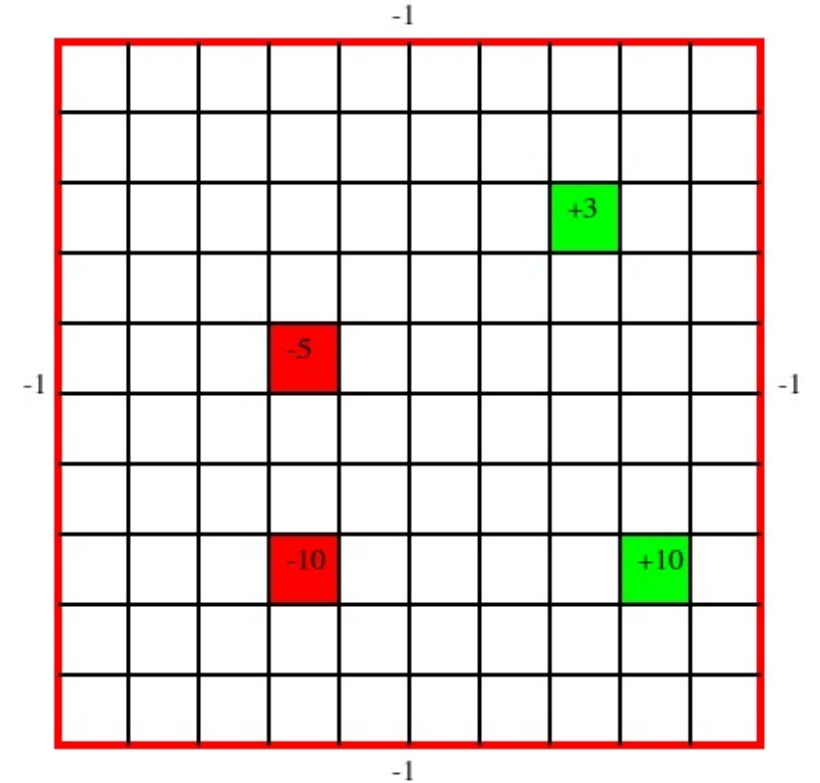


Introduction

Collective Adaptive Systems (CAS): Large-scale networks of autonomous agents (e.g., drone swarms) operating in dynamic, uncertain environments

Challenge: Traditional Reinforcement Learning (RL) relies on *Reward Shaping*:

- **Manual & Error-prone:** Hard to design for complex behaviors
- **Opaque:** Lacks formal guarantees or interpretability



Formal Methods improve Traditional RL

How? Use **STREL** (Spatio-Temporal Reach and Escape Logic) as **Reward Function**

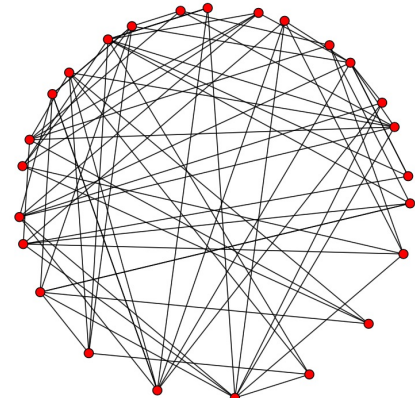


Spatio-Temporal Reach and Escape Logic STREL

STREL: a formal logic used to specify and verify the behavior of distributed systems evolving over both **time** and **space**

The system is modeled as a **dynamic weighted graph**:

- **nodes** represent agents or locations
- **edges** represent interactions, proximity, or connectivity



Semantics:

- **Boolean**: tells whether a formula is true or false
- **Quantitative**: measures how strongly a property is satisfied or violated through robustness

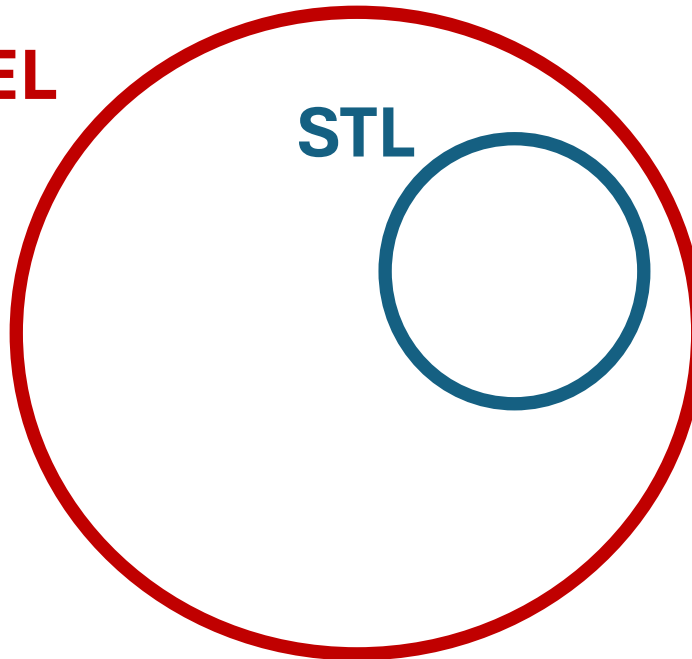
Syntax

STREL combines: **logical, temporal** operators and **spatial** operators

$$\varphi := \mu \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \mathbf{U}_{[t_1, t_2]} \varphi_2 \mid \varphi_1 \mathbf{S}_{[t_1, t_2]} \varphi_2 \mid \varphi_1 \mathcal{R}_{[d_1, d_2]}^{f:A \rightarrow B} \varphi_2 \mid \mathcal{E}_{[d_1, d_2]}^{f:A \rightarrow B} \varphi$$

STREL

STL



A Differentiable STREL Library

Goal: to integrate formal logic monitoring within modern machine learning workflows

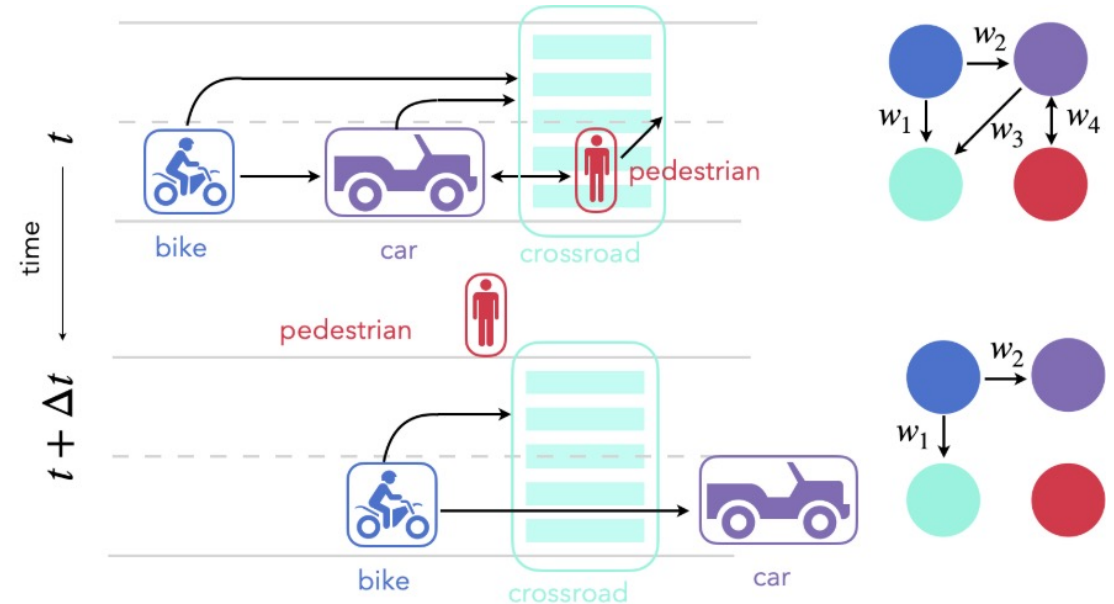
Differentiability

Tensorized Implementation

Performance Efficiency → GPU

parallelization

} PyTorch



Colored STREL (CSTREL): STREL quantitative semantics is ill-defined for discrete node attributes

- Each node has a label (color), used as a **filter** during monitoring
- Atomic predicates and spatial operators restricted to specific node types



Policy Optimization

CSTREL for task specification ➡ Policy Optimization

Goal: to determine the policy **parameters** θ that maximize the robustness of the **STREL** specification ϕ over the trajectories induced by the policy:

$$\theta^* = \arg \max_{\theta} \rho(\phi, \tau_{\theta})$$



$\rho(\phi, \tau_{\theta})$: quantitative robustness of trajectory τ with respect to ϕ



Reward Shaping

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{b=1}^B \left(\frac{1}{T} \sum_{t=1}^T \min_{d \in D} \rho_{b,d,t} \right) + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}$$

↓

Regularization term
on actions

$\rho_{b,d,t}$ is the quantitative robustness
of an agent d (D is the set of
agents) at time t in environment b

Minimizing this term is equivalent to **maximizing the STREL robustness**



Experimentals Setup



A group of simulated battery-powered **drones** navigates in an area and need to accomplish some specific **tasks**



Graph Structure:

The environment is represented as a **fully-connected graph**

- each edge is weighted by *Euclidean distance*
- each node is described by a 6-dimensions *feature* vector: $(x, y, v_x, v_y, b, \gamma)$

node position

Experimentals Setup



A group of simulated battery-powered **drones** navigates in an area and need to accomplish some specific **tasks**



Graph Structure:

The environment is represented as a **fully-connected graph**

- each edge is weighted by *Euclidean distance*
- each node is described by a 6-dimensions *feature* vector: $(x, y, \underbrace{v_x, v_y}_{\text{velocity}}, b, \gamma)$

Experimentals Setup



A group of simulated battery-powered **drones** navigates in an area and need to accomplish some specific **tasks**



Graph Structure:

The environment is represented as a **fully-connected graph**

- each edge is weighted by *Euclidean distance*
- each node is described by a 6-dimensions *feature* vector: $(x, y, v_x, v_y, b, \gamma)$



battery level

Experimentals Setup



A group of simulated battery-powered **drones** navigates in an area and need to accomplish some specific **tasks**



Graph Structure:

The environment is represented as a **fully-connected graph**

- each edge is weighted by *Euclidean distance*
- each node is described by a 6-dimensions *feature* vector: $(x, y, v_x, v_y, b, \gamma)$

node type
label
(CSTREL)

Experimentals Setup



A group of simulated battery-powered **drones** navigates in an area and need to accomplish some specific **tasks**



Graph Structure:

The environment is represented as a **fully-connected graph**

- each edge is weighted by *Euclidean distance*
- each node is described by a 6-dimensions *feature* vector: $(x, y, v_x, v_y, b, \gamma)$

↳ 3 types of nodes (*label encoding* **CSTREL**):

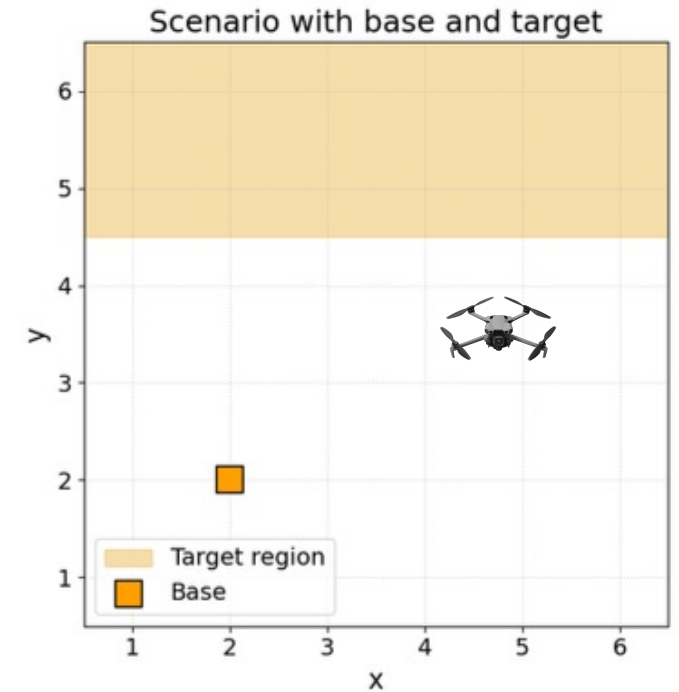
- **Base** ($\gamma = 0$): charging station (*static*)
- **Drone** ($\gamma = 1$): mobile agent (*dynamic*)
- **Grid** ($\gamma = 2$): spatial reference (*static*)



Initialization

At the beginning of each rollout:

- **Grid** nodes and **base** node are placed at fixed positions
- **Drones** positions are sampled *randomly* inside the grid with $b = 1.0$, and $v_x, v_y = 0$



Dynamics Model

Policy $\rightarrow$ 2D action $\rightarrow$ velocity update $\rightarrow$ new position



Drone motion is **continuous**, bounded to the spatial domain, and smoothed by inertia

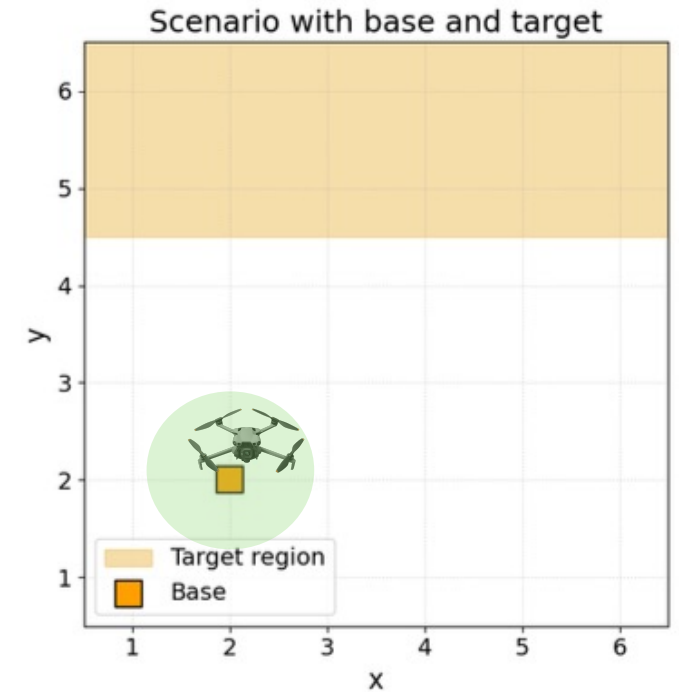
Battery Dynamics



Initialization

At the beginning of each rollout:

- **Grid** nodes and **base** node are placed at fixed positions
- **Drones** positions are sampled *randomly* inside the grid with $b = 1.0$, and $v_x, v_y = 0$



Dynamics Model

Policy $\rightarrow$ 2D action $\rightarrow$ velocity update $\rightarrow$ new position



Drone motion is **continuous**, bounded to the spatial domain, and smoothed by inertia

Battery Dynamics



2%



10%

- Drone nodes
- Grid nodes
- Base node

Battery Recharge - No Collision

If a drone has high battery, it should reach the target region within a given time horizon

$$\phi_1 = \text{high_battery} \Rightarrow F_{[0,t]}(\text{alive Reach}_{[d_1,d_2]} \text{at_target})$$



- Drone nodes
- Grid nodes
- Base node

Battery Recharge - No Collision

If a drone has high battery, it should reach the target region within a given time horizon

$$\phi_1 = \text{high_battery} \Rightarrow F_{[0,t]}(\text{alive Reach}_{[d_1,d_2]} \text{at_target})$$

If a drone has low battery, it should reach a charging base and remain there until sufficiently recharged

$$\phi_2 = \text{low_battery} \Rightarrow F_{[0,t]} \left((\text{alive Reach}_{[d_1,d_2]} \text{free_base}) U \text{full_battery} \right)$$



- Drone nodes
- Grid nodes
- Base node

Battery Recharge - No Collision

If a drone has high battery, it should reach the target region within a given time horizon

$$\phi_1 = \text{high_battery} \Rightarrow F_{[0,t]}(\text{alive Reach}_{[d_1,d_2]} \text{at_target})$$

If a drone has low battery, it should reach a charging base and remain there until sufficiently recharged

$$\phi_2 = \text{low_battery} \Rightarrow F_{[0,t]} \left((\text{alive Reach}_{[d_1,d_2]} \text{free_base}) U \text{full_battery} \right)$$

A drone should remain alive at all times, meaning keep the battery level non-negative

$$\phi_3 = \text{alive}$$



- Drone nodes
- Grid nodes
- Base node

Battery Recharge - No Collision

If a drone has high battery, it should reach the target region within a given time horizon

$$\phi_1 = \text{high_battery} \Rightarrow F_{[0,t]}(\text{alive Reach}_{[d_1,d_2]} \text{at_target})$$

If a drone has low battery, it should reach a charging base and remain there until sufficiently recharged

$$\phi_2 = \text{low_battery} \Rightarrow F_{[0,t]} \left((\text{alive Reach}_{[d_1,d_2]} \text{free_base}) U \text{full_battery} \right)$$

A drone should remain alive at all times, meaning keep the battery level non-negative

$$\phi_3 = \text{alive}$$

The drones should not be closer than d_{safe}

$$\phi_4 = \|p_1(t) - p_2(t)\| > d_{safe}$$

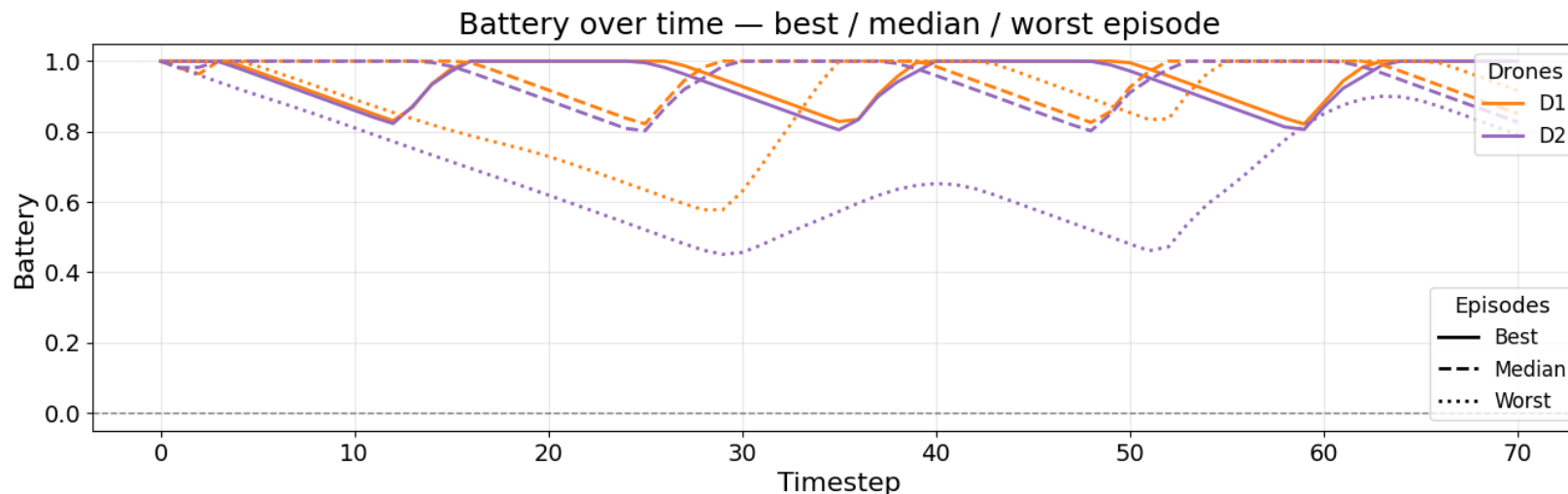
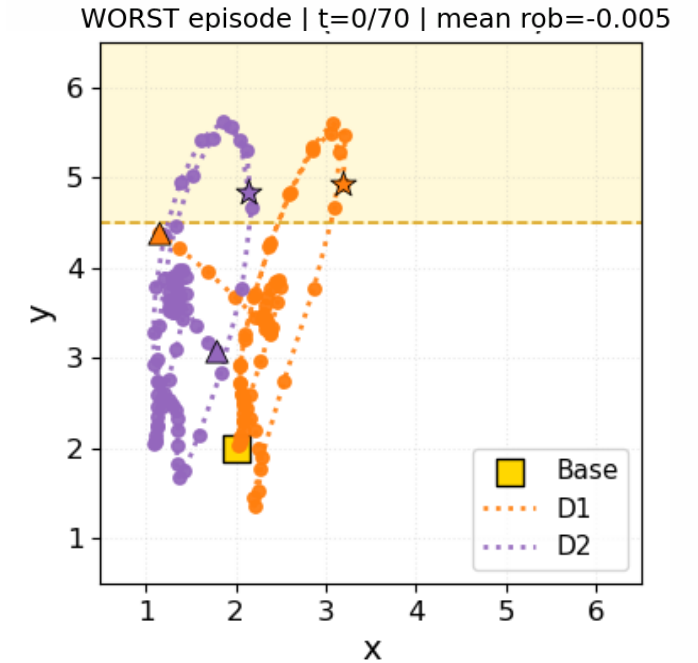
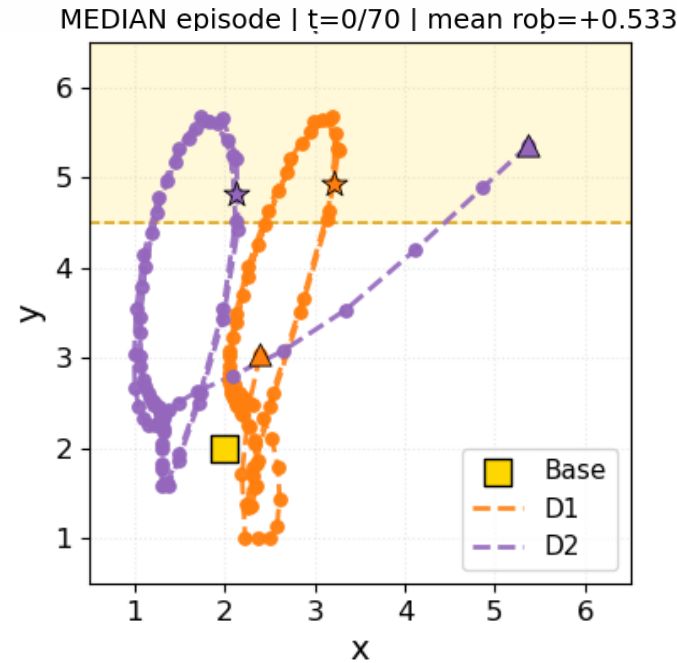
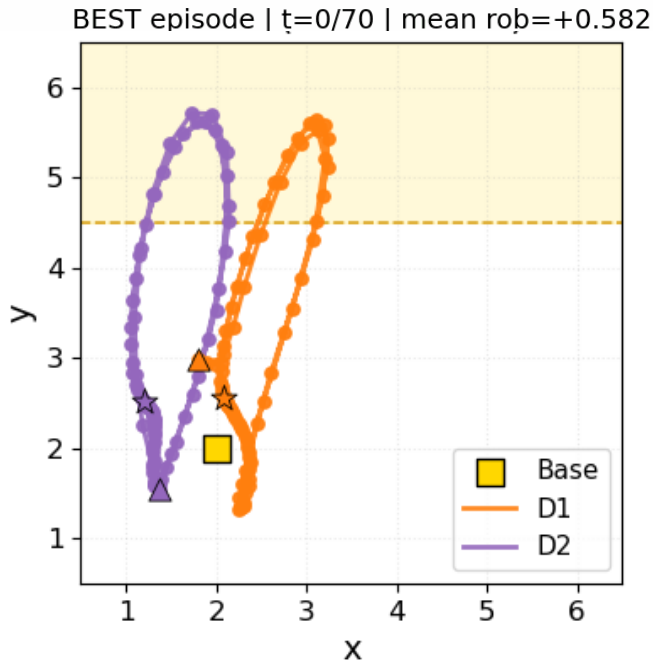


Battery Recharge - No Collision

$$\Phi = G_{[0,T]}(\phi_1 \wedge \phi_2 \wedge \phi_3 \wedge \phi_4)$$



Battery Recharge - No Collision



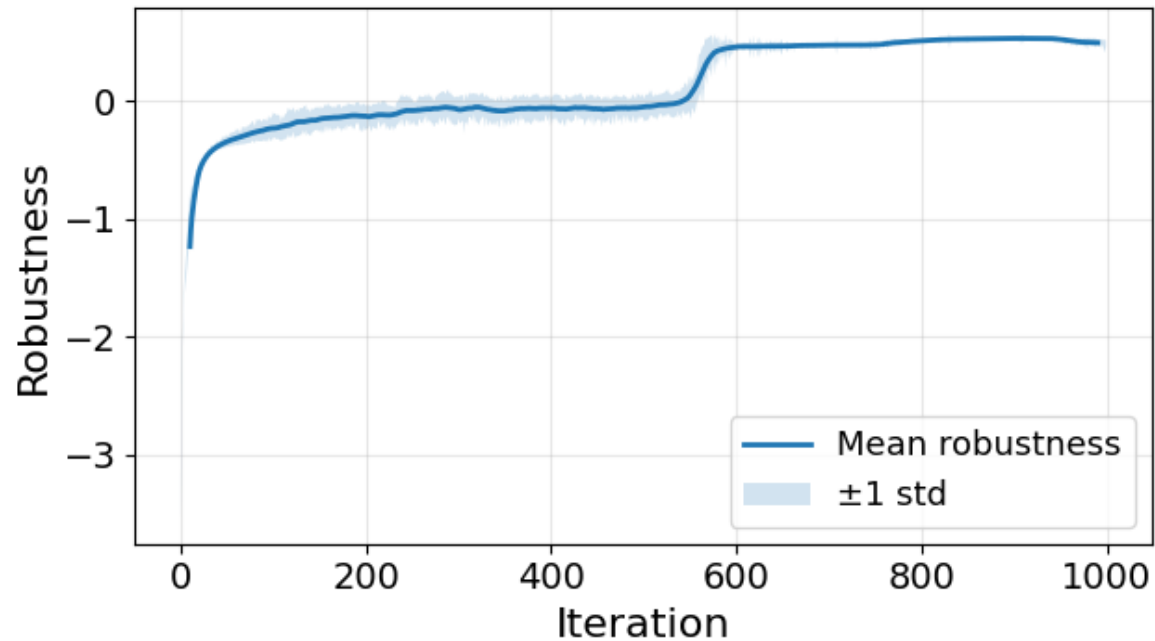
On average 0.02 collisions per episode are observed, with a maximum of 3 collisions in a single episode



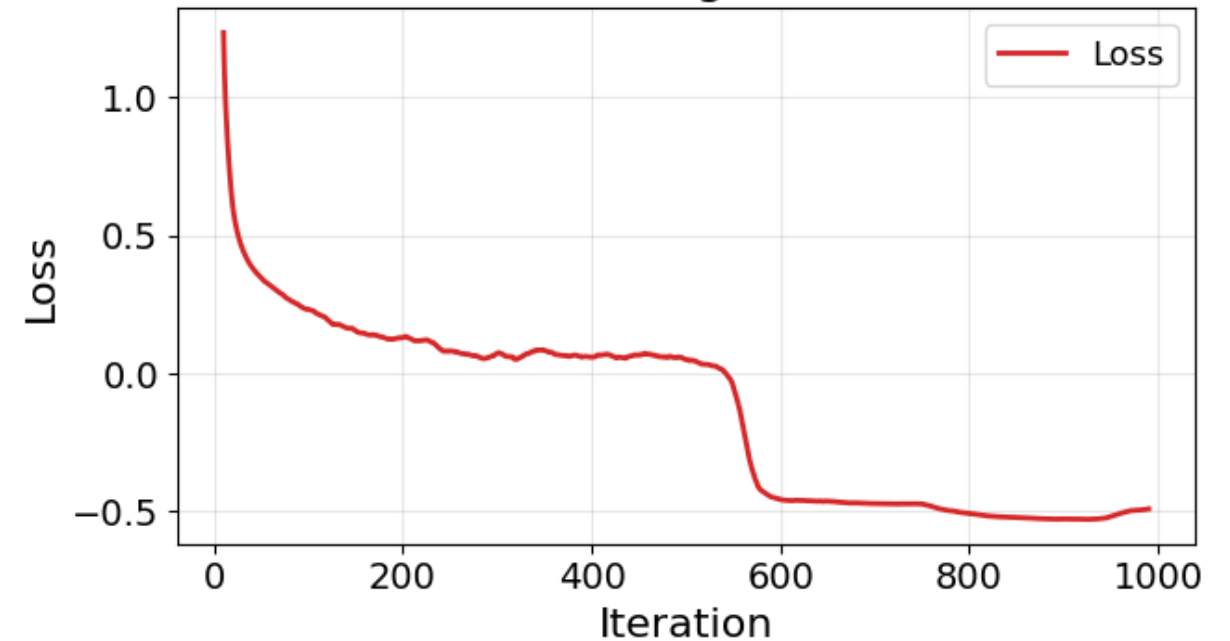


Battery Recharge - No Collision

Robustness over iterations



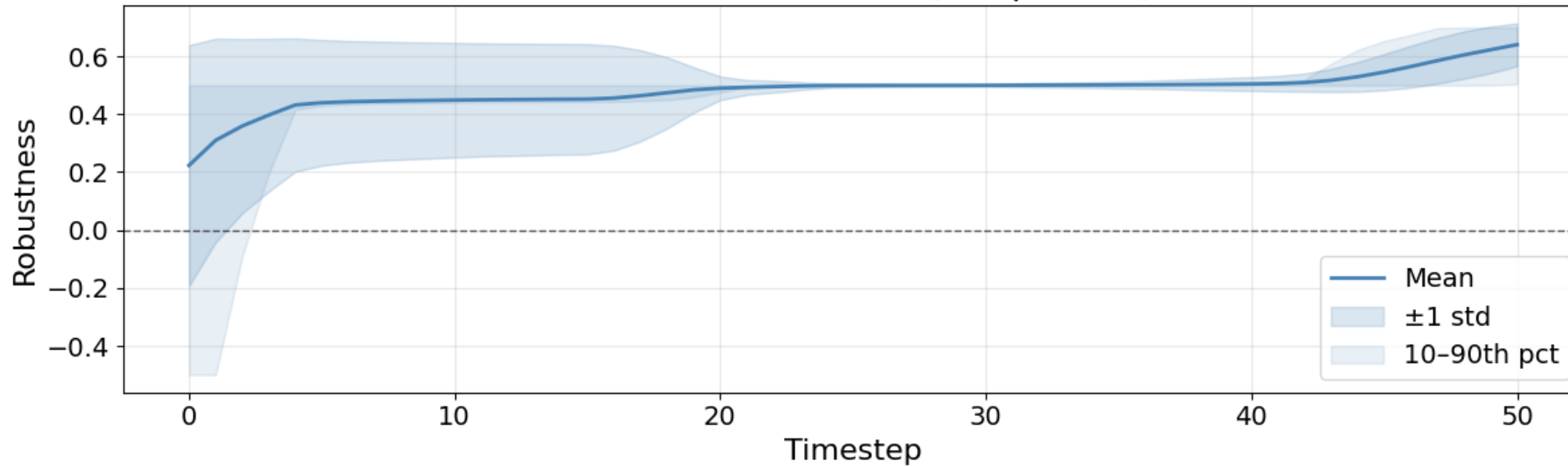
Training loss



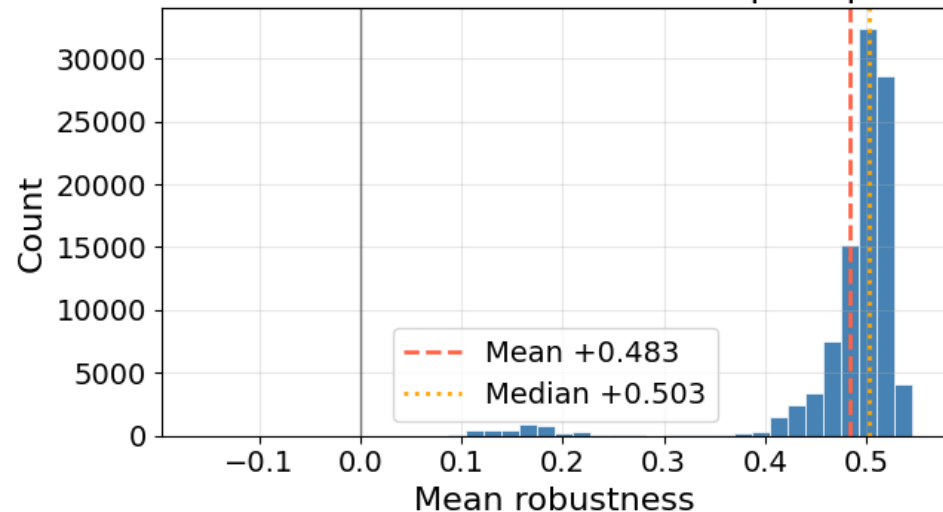


Battery Recharge - No Collision

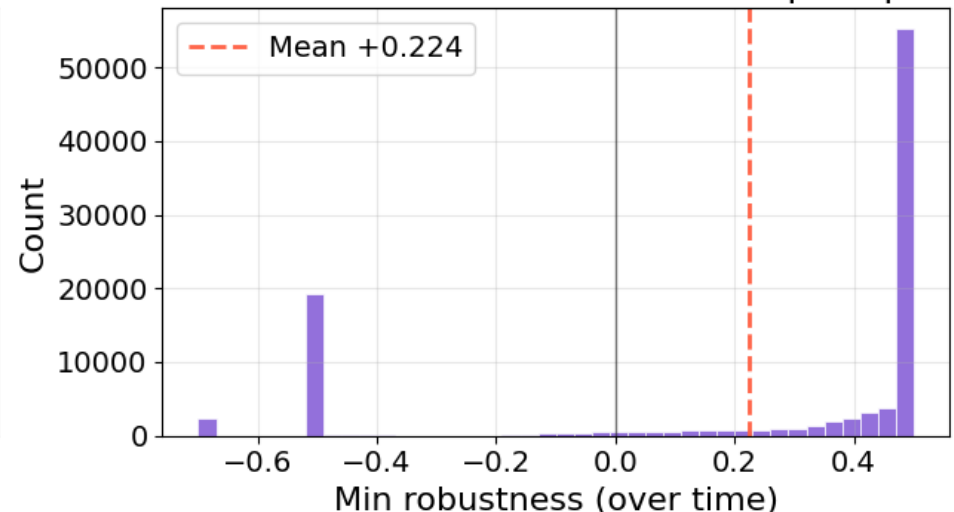
Robustness over time (all episodes)



Distribution of mean robustness per episode



Distribution of minimum robustness per episode



Conclusions

- Formal spatio-temporal logic can guide policy optimization in collective adaptive systems
- STREL provides a natural way to encode mission, safety, and resource constraints
- Differentiable robustness makes these specifications usable as a learning objective

Future works:

- Extending the framework to larger graphs
- Scaling to a higher number of agents



**thank you for your
attention!**

